```
        torch.nn.utils.clip_grad_norm_(GWagent.parameters(),
max_norm=1.0) 8
        opt.step()
    if i % update_freq == 0:
9
        Tnet.load_state_dict(GWagent.state_dict())
```

- ***1*** Sets up environment
- ***2*** Creates main relational DQN
- ***3*** Creates target DQN
- ***4*** Sets the maximum steps before game will end
- ***5*** Creates the experience replay memory
- ***6*** Uses an epsilon-greedy policy for action selection
- ***7*** Rescales the reward to be slightly negative on nonterminal states
- ***8*** Clips the gradients to prevent overly large gradients
- ***9*** Synchronizes the main DQN with the target DQN every 100 steps

Our self-attention double DQN reinforcement learning algorithm will learn how to play fairly well after about 10,000 epochs, but it may take up to 50,000 epochs before it reaches maximum accuracy.

Figure 10.22 shows the log-loss plot we get, and we also plotted the average episode length. As the agent learns to play, it should be able to solve the games in fewer and fewer steps.

Figure 10.22. Top: Log-loss plot during training. The loss drops quickly in the beginning, increases a bit, and then very slowly begins decreasing again. Bottom: Average episode length. This gives us a better idea of the performance of the agent since we can clearly see it is solving the episodes in a shorter number of steps during training.